SHOPEASE · FLUTTER SHOPPING APP

Technology Used

What the app is built with, and how each piece is used.

1. At a glance

Area	Choice	How it is used
Framework	Flutter 3.41 · Dart 3.11	One codebase, built for Android. Material 3 widgets.
Architecture	MVC	Models (data), Views (screens), Controllers (logic and state), plus a Services layer for data access.
State management	`provider`	Each controller is a `ChangeNotifier`; screens listen and rebuild when it calls `notifyListeners()`.
Font	Poppins (`google_fonts`)	Set once in the theme, so every text in the app uses it.
Product data / API	DummyJSON (free, no key)	One HTTP request fetches all products with images, prices and ratings.
Networking	`http`	A single GET request with a 15-second timeout.
Local storage	`shared_preferences`	Saves the login session, cart, wishlist, orders, recent searches and theme on the phone.
Images	`cached_network_image`	Downloads each picture once, then shows it from the cache.
Loading effect	`shimmer`	Grey animated placeholders while products load.
Testing	`flutter_test`	13 unit tests (validators, accounts, per-account data, cart, theme, search/sort, pagination).

2. Architecture (MVC + Provider)

Folder (`lib/`)	Role
`models/`	**M** · plain data classes: `Product`, `ShopCategory`, `CartItem`, `Order`, `AppUser`.
`views/`	**V** · the screens: splash, login, home, categories, search, product, cart, wishlist, profile, settings.
`controllers/`	**C** · state and logic: Auth, Catalog, PagedProducts, Cart, Wishlist, RecentSearch, Nav, Theme.
`services/`	The only code that reaches data: `ProductRepository` (API), `StorageService` (phone storage), `ProductQuery` (search/filter/sort rules), `MockData` (offline fallback).
`widgets/`	Reusable UI: product card, paged product grid, buttons, text field, filter sheet, shimmer, empty states.
`core/`	Theme, colours, routes, validators, responsive helpers, demo accounts.

```
Screen tap → Controller method → Service (API or storage) → Controller updates state →
notifyListeners() → Provider rebuilds the screen
```

Controllers are created once in `main.dart` with `MultiProvider`. Screens read them with `context.watch` (rebuild on change) or `context.read` (just call a method).

3. Data and API

Where the product data comes from

`ProductRepository` calls `GET https://dummyjson.com/products?limit=0&select=...`. `limit=0` returns every product in one response. Each item is turned into a `Product` object; vehicle categories are dropped, leaving 22 categories grouped into 6 app categories.

- The result is cached in memory, so every screen shares one list.
- The list is shuffled with a fixed seed (7): the feed looks mixed but is the same on every launch.
- If the request fails, about 50 built-in placeholder products are used and Home shows an "Offline" note.

What each value is based on

Value	Source
Name, description, brand, price, discount %, rating, stock, images	From the API response.
"Original" (struck-through) price	Calculated: price ÷ (1 − discount%).
Review count	Generated from the product id (the API has no review counts).
Colour and size options	The API has no colour or size data, so every product shows one default choice: Color "Default" and Size "One size".
Best Deals, category counts and covers	Calculated from the loaded products.
Search, sort, price and category filters	Done on the phone by `ProductQuery`.
Banners, popular searches, notification text	Fixed text in the code.

4. Storage (on the phone)

`shared_preferences` keeps small key-value data. Cart, wishlist, orders and searches are stored **per account**: every key starts with the account's e-mail.

Key	Holds	Scope
`<email>:cart`	Cart lines (a product copy and quantity)	Per account
`<email>:orders`	Placed orders	Per account
`<email>:wishlist`	Liked product ids	Per account
`<email>:recent`	Recent search words (up to 8)	Per account
`user`	Logged-in user (name, e-mail, phone)	Global
`theme_mode`	Light, dark or system	Global

5. Login and accounts

There is no backend. Three accounts are fixed in `core/demo_accounts.dart` and accept the e-mail or phone number. A one-second delay imitates a network call. Only the logins are fixed; every account starts with an empty cart and wishlist.

Name	Email	Phone	Password
Salman Khan	salman@gmail.com	9876543210	Shop@123
Aisha Rahman	aisha@shopease.com	9123456780	Aisha@456
Rahul Nair	rahul@shopease.com	9000011111	Rahul@789

6. Design, font and theme

- **Font:** Poppins, loaded by `google_fonts` and applied to the whole text theme. It is downloaded the first time it is needed, so the first launch needs internet; without it the system font is used.
- **Colours:** `AppColors` holds the brand blue `#1F64F2` and status colours as constants; background, card, border and text colours are getters that return a light or dark value.
- **Theme:** `AppTheme.build()` creates the Material 3 theme. Light, Dark and System are chosen in Settings and saved by `ThemeController`.
- **Responsive:** product grids use 2 to 5 columns depending on width; forms scroll so the keyboard never hides a field.

7. Other packages and tools

Item	Used for
Material icons	All icons in the app (no extra icon package).
`flutter analyze`	Static checks; no issues at the last run.
Android emulator (Pixel 6a)	Used to run and check the app.